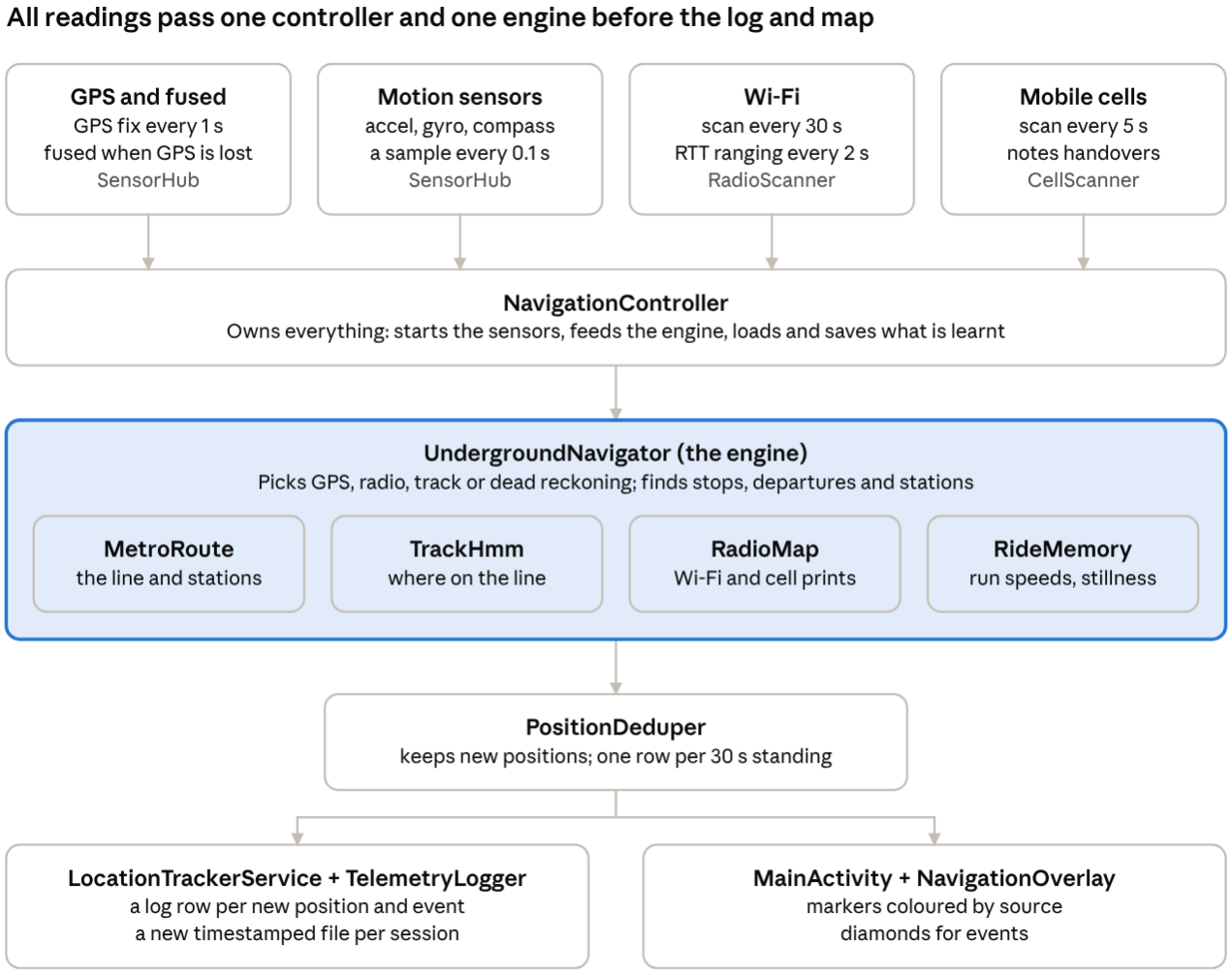
Metro Navigator — Code Flow Guide

Nov 2, 2026 · @kamlesh kumar Patel

Metro Navigator works out a train's position underground on the phone alone: the sensors feed one engine every 0.1 s, the engine keeps the position on the metro line and corrects it at every station stop, and each position is logged and drawn on the map.

At a glance



code flow · sensors to controller to engine to log and map

Four kinds of sensor feed one controller; the engine inside it decides the position and the source; only new positions go on to the log file and the map.

How to explain it in two minutes

The app answers one question inside a tunnel — where is the train now? — with no server and no internet.

1. **Before going underground** the phone already holds the city's metro lines (18 Indian cities are built in; any other city downloads once) and uses GPS where it has it.
2. **The phone is mounted on the train.** Its motion sensors feel every push: pulling away, cruising, braking, standing still.
3. **Without GPS** the position moves **along the line** at the speed worked out from those pushes, so it can never leave the track.
4. **At every stop** the app checks the train really braked, works out from the distance travelled which station it is, and snaps the position there. The error resets at every station.
5. **Wi-Fi and mobile cells** heard in stations and tunnels are learnt on each ride and correct the position on later rides; a cell handover is a checkpoint.
6. **Every position** is written to a log file (a new one each time logging starts) with its source and ± error, and drawn on the map.

One line: *the line map and the motion sensors carry the position, station stops reset it, and Wi-Fi and cells learnt on earlier rides correct it.*

Startup: from opening the app to the first position

One object, `NavigationController`, owns everything; the screen and the logging service only switch it on and off.

1. **`MainActivity.onCreate`** sets up the map (OpenStreetMap tiles, the app's identity for the tile server), builds the screen and calls `NavigationController.init`.
2. **`NavigationController.init`** (once per app process) creates `SensorHub` (GPS + motion sensors), `RadioScanner` (Wi-Fi + Wi-Fi RTT), `CellScanner` (mobile cells) and `MetroRouteService` (line data). In the background it:
   - loads the saved metro lines and imports the 18 bundled cities on first run;
   - loads `radio_map.txt` and `ride_memory.txt` (radio scans wait until this is done, so nothing learnt is overwritten);
   - listens for a network, to download lines when one appears.
3. **`MainActivity.onStart`** calls `NavigationController.acquire("activity")`, which starts the sensors (needs location permission; otherwise the warning bar asks for it):
   - `SensorHub.start` — GPS every 1 s, fused location every 2 s, accelerometer, orientation and gyroscope about 50 times a second, barometer, step detector;

- `RadioScanner.start` — Wi-Fi scan every 30 s, RTT ranging every 2 s;
- `CellScanner.start` — mobile cells every 5 s.

4. **First GPS fix** → `UndergroundNavigator.onLocationFix` sets the position. `maybeLookupLines` finds the nearest saved line within 400 m and calls `setRoute`, so the train is "on the line". No line here and a network available → the city's lines (30 km) download from OpenStreetMap.
5. **Start logging** → `LocationTrackerService` (a foreground service with a notification): holds a wake lock so sensors keep running with the screen off, acquires the controller as `"service"`, opens a new log file (`TelemetryLogger.startSession`) and writes every position and event.
6. **Screen closed** → the activity releases its hold; while logging, the service keeps everything running. **Stop logging** → the service writes "Logging stopped", closes the file and releases. When nothing holds the controller, the sensors stop and the learnt maps are saved.

The 0.1-second loop: what one motion reading does

Ten times a second the sensors produce one reading, and the engine moves the position by it.

1. **`SensorHub`** reads the accelerometer (gravity removed), orientation sensor, gyroscope and step detector about 50 times a second. Every 100 ms it hands over one `MotionSample`: the push east and north, *still / walking / moving*, how long it has been still, steps, the phone's heading, turn rate and vibration level.
2. **`NavigationController.onMotionSample`** passes it to the engine, shows the new state on screen (at most 4 times a second) and, without GPS, writes a position every 2 s (a log row; a map marker only if it moved).
3. **`UndergroundNavigator.onMotion`** — the engine:
   - **With GPS** it only learns: the compass error (from GPS speed changes), how this phone vibrates standing vs moving, the gyroscope's drift. The GPS fix is the position.
   - **Without GPS**, in this order:
     a. *Still?* The learnt vibration threshold can turn "moving" into "standing"; the braking check then confirms a stop or calls it a smooth glide.
     b. *Standing* → speed 0 and the sensor offsets are re-learnt; still 5 s on a line → the station decision (`handleStop`).
     c. *Walking* → on a line the train keeps its speed; with no line, steps × step length along the phone's heading.

   d. *Moving* → after a stop, wait for a real pull-away; then the speed comes from the push along the track, pulled toward the learnt speed of this stretch and cross-checked with the speed added up since the last stop.
   e. *Position* → on a line: further along the track, waiting at the next station until the train stops there (unless it runs through), with a curve check from the gyroscope. No line: forward along the heading, turned by the gyroscope.
   f. *Error* → the ± error grows with how unsure the speed is.
4. **Radio scans** arrive on their own timers (Wi-Fi every 30 s, cells every 5 s). With a known position they are learnt; without GPS on a line they correct the position through a probability model of where on the track the train can be (`TrackHmm`).

With and without GPS: what the engine does in each situation

Dead reckoning (the motion sensors) carries the position whenever GPS is missing; every other source only corrects it, so nothing jumps when a source drops out.

Situation	Log `Source`	What happens	Corrected by
GPS good (satellite fix, ± 50 m or better)	`GPS`	The fix is the position. The engine learns speed, compass error, sensor offsets, vibration levels, Wi-Fi and cell places.	—
GPS just lost (no fix for 6 s)	`TRACK` or `DR`	The 6 s gap is covered at the last speed; event "GPS lost".	—
No GPS, on a metro line or test route	`TRACK`	Moves along the line by the motion sensors; snaps to the station at each stop.	Station stops, Wi-Fi / cells / RTT, curves, network fixes
No GPS, no line	`DR`	Moves along the heading (turned by the gyroscope) at the sensor speed.	Network fixes, learnt Wi-Fi / cell places
No GPS, network location only	`NET` when it dominates	A network fix corrects the estimate after a consistency check; wild jumps are ignored.	—

Situation	Log `Source`	What happens	Corrected by
No position at all yet, underground	`RADIO` or `NET`	The first position comes from learnt Wi-Fi / cells or a network fix, then locks onto the nearest line.	—
GPS comes back	`GPS`	Trusted after one fix of ± 15 m or two that agree; event "GPS recovered … estimate was off by N m".	—
Tunnel test (test drive)	`TRACK`	GPS and network are hidden from navigation; GPS only measures the error.	Station stops, Wi-Fi / cells, curves

Trust order when they disagree: GPS → station stop + distance travelled → Wi-Fi RTT, Wi-Fi and cell fingerprints, handovers → network location → dead reckoning alone.

Stops, departures and speed: how the motion sensors find them

A stop needs both stillness and braking; a departure needs a one-way push; speed is the push added up, kept honest by the learnt speed of each stretch.

- **Still, walking or moving** (`SensorHub` , from the last ~2 s): *still* = vibration under 0.06 m/s² (or under this phone's learnt threshold) with almost no rotation, for 1 s; *walking* = 3 steps in 3 s.
- **Stop** (`UndergroundNavigator`):
  1. Feels still → did it brake? The speed added up since the last stop must be back near zero. Yes → a stop, speed 0. No → a smooth glide, the speed is kept. Still for 15 s → a stop anyway.
  2. Stopped 5 s on a line → which station? The distance travelled since the last station names the station at that distance; otherwise the nearest one, if it is likelier than a signal stop. The position snaps there and the error drops to ± 40 m. No station near → "Train stopped between stations (signal?)".
  3. While standing, the accelerometer offset and gyroscope drift are re-learnt.
- **Departure**: the speed change over the last 6 s reaches 1.5 m/s with the pushes in one direction (a jiggled phone pushes both ways and cancels out). That direction gives the direction of travel on the line when unknown, and corrects the compass. Twenty

seconds of "moving" without one → movement resumes anyway.

- **Speed**, every 0.1 s: the push along the track × 0.1 s is added (at most ± 2.5 m/s²), pulled toward the expected speed of this stretch (learnt from GPS, station-to-station times and earlier rides) and blended with the speed added up since the last stop.
- **Running through a station**: the position waits at the next station; if 6 s later the train still runs at 8 m/s or more with no braking, it moves on ("Passed X without stopping").

Sensors and data sources: what is used, how often, and when

Twelve inputs, all on the phone; only the line download and network location ever use the internet.

Input	Read by	How often	Used for	When	Needs
GPS receiver	`SensorHub`	1 s	The position; teaches speed, compass error, offsets	Above ground	Precise location
Fused / network location	`SensorHub`	2 s	Gentle correction	Only while GPS is lost	Play services, internet
Accelerometer (gravity removed)	`SensorHub`	~50 Hz	Pushes: speed, stops, departures, vibration	Always	—
Orientation sensor	`SensorHub`	~50 Hz	Turns phone axes into east / north	Always	—
Gyroscope	`SensorHub`	~50 Hz	Curve checks on a line; heading with no line; stillness	Always	—
Compass (inside orientation)	`SensorHub`	~50 Hz	Rough direction of travel; its error is learnt	Departures	—
Barometer	`SensorHub`	~5 Hz	Quick pressure change = going underground	Stairs, ramps	A barometer
Step detector	`SensorHub`	Each step	Walking vs train moving	Always	Physical activity permission
Wi-Fi scan	`RadioScanner`	30 s	Wi-Fi fingerprints of stations and tunnels	Learnt with a known position; used without GPS	Precise location, Wi-Fi scanning on

Input	Read by	How often	Used for	When	Needs
Wi-Fi RTT	`RadioScanner`	2 s	Distance to learnt access points (metres)	Without GPS	Android 9+, Nearby devices (13+)
Mobile cells	`CellScanner`	5 s	Cell fingerprints and handover checkpoints	Learnt with a known position; used without GPS	Precise location, location on
Metro line map	`MetroRouteService`	On start; new area	Keeps the position on the track; station positions	Always	Internet once per city (18 built in)

The phone's satellite count and the network state are also read, for the screen and the log only.

Radio: Wi-Fi, mobile cells and handovers

The phone builds its own map of what it hears where (`RadioMap`), learning only where the position is certain, and uses it only without GPS.

	Wi-Fi	Mobile cells	Handover
What is heard	Access points and their strength	Serving and neighbour cells (4G, 5G, 3G, 2G) and their strength	The serving cell changing, e.g. cell A → cell B
Comes from	`RadioScanner` every 30 s	`CellScanner` every 5 s	`CellScanBuilder` , marked for 4 s after a change
Learnt when	Good GPS (± 20 m); standing at a confirmed station; along a tunnel after a ride between two confirmed stations	Same	Same
A match needs	2 access points in common	1 cell in common	The same handover heard there before
A missing loud one	Counts against the place	Counts against the place	Says nothing

	Wi-Fi	Mobile cells	Handover
Weight of one scan	Full (0.35)	Lower (0.25), as scans come 6× as often	As a cell scan
Needs internet	No	No	No

How a scan corrects the position: `UndergroundNavigator.onRadioScan` turns the matches into a likelihood for every 10 m of the line, `TrackHmm` combines it with where the motion model says the train is, and the position moves only when the result is clearly sharper or the estimate is clearly elsewhere (a train never moves backwards for a small correction). Wi-Fi RTT ranges add a metre-level distance to learnt access points. A cold start with no position at all uses `RadioMap.locate` to place the train first.

What the phone learns and saves

Everything learnt stays on the phone, in the app's own storage, and is written through a temporary file so a crash never leaves a half-written one.

What	Learnt from	Saved in	Written when	Used when
Metro lines and stations	OpenStreetMap download; 18 cities bundled in the app	`metro_routes/*.json` (+ `downloaded_areas.json`, `bundled_imported.txt`)	After a download or the first-run import	Always
Wi-Fi and cell fingerprints, handovers	Scans heard where the position is certain	`radio_map.txt`	At most once a minute while it changes; when tracking stops	Without GPS
Speed of each station-to-station stretch	Moving time between two confirmed stations	`ride_memory.txt`	Same as above	The next ride on that stretch
Phone vibration standing vs moving	GPS speed under 0.3 m/s vs over 4 m/s	`ride_memory.txt`	Same as above	Stop detection without GPS
Test routes	A first test drive with GPS	`test_routes/*.json`	On Finish	Drive again
Map tiles	Tiles shown while online	App cache	As tiles load	Map background, offline where seen before
Log	Every position and event	`Download/TestGPSLocation/Metro_Navigator_telemetry_<date>_<time>.txt`	Each row as it happens	Analysis after a ride

Re-learnt every session, not saved: the compass error, the accelerometer offset, the gyroscope drift and the typical speed of the line.

Log file and map: when a row or marker is written

A row is written only when the position changes (at the log's 6 decimals), at most every GPS fix with GPS and every 2 s without, and at least once every 30 s; each changed position also gets a map marker.

- **Log rows**: one per new position (`Lat/Lon …`), one per event (`EVENT: Stopped at X`, `GPS lost`, `Departure detected`, `Passed X without stopping`, `GPS recovered … off by N m`, test-drive messages), plus `Logging started` / `Logging stopped`. A tunnel test writes a row every 2 s even when the position stands, because the error still changes.
- **A new file** each time logging starts, named after that moment; `LocationTrackerService` writes it through `TelemetryLogger` on a background thread.

Column	Meaning
`[2026-11-02 15:52:27.969]`	Phone time of the position
`Lat, Lon`	Estimated position
`Source`	Who is in charge: `GPS`, `TRACK` (on the line), `RADIO` (Wi-Fi / cell), `NET` (network), `DR` (dead reckoning)
`Speed`	Estimated speed (m/s)
`GPS` / `Network`	Satellite fix `Acquired` or `Lost`; internet `Connected`, `Weak` or `Lost`
`Error`	Estimated ± error (m)
`GPS accuracy`, `Satellites used`	From the GPS receiver
`Line`, `Next`	Metro line followed and next station
`TUNNEL TEST real GPS … Error`	Tunnel test only: the hidden GPS position and the distance to the estimate (m)
`Sensors`	Still / walking / moving, vibration (m/s²), turn rate (°/s)
`Cell`	Serving mobile cell and its strength (dBm)

Map (`NavigationOverlay`): grey line and white dots = the metro line or test route and its stations (names from zoom 14); small dots = logged positions, coloured by source (green GPS, blue track, purple network, teal Wi-Fi / cell, orange dead reckoning); orange

diamonds = events (tap for details); white line and ring = the real GPS in a tunnel test; large dot = the current position with its ± circle and direction arrow. The last 10,000 markers are kept; ⋮ › **Clear map markers** empties the map, not the log.

Files: every source file, its job and when it runs

Paths below are under `app/src/main/java/com/example/testgpslocation/` unless the module is named; line counts are approximate.

Screen and service

File	Job	When it runs
`MainActivity.kt` (662)	The screen: map, compact panel, menu, line picker, test-drive dialog and bar, logging button, permission and battery prompts	While the app is on screen; it only reads the controller's flows and forwards taps
`service/LocationTrackerService.kt` (161)	Foreground service: keeps tracking alive with the screen off (wake lock), writes every position and event to the log file	From **Start tracking** to **Stop**, screen on or off
`map/NavigationOverlay.kt` (236)	Draws the line, stations, markers coloured by source, event diamonds, the test-drive truth line and the current position	On every map redraw

Engine (the brain)

File	Job	When it runs
`engine/NavigationController.kt` (635)	One owner for the whole process: starts sensors and scanners, feeds the engine, loads and saves what is learnt, publishes state to the screen and service, runs test drives	The whole time tracking is on
`engine/UndergroundNavigator.kt` (1743)	The engine: chooses GPS, track, radio, network or dead reckoning; detects stops and departures; snaps to stations; follows curves; learns biases	On every GPS fix, every 0.1 s motion sample and every radio scan
`engine/MetroRoute.kt` (212)	A line: track points, stations, projecting a point onto the line, distance along it	Whenever a position is put on the line
`engine/MetroRouteService.kt` (300)	Offline line data: lines bundled in the APK first, then the cache, then a download from OpenStreetMap (Overpass)	When a line is chosen or refreshed
`engine/OverpassParser.kt` (99)	Turns OpenStreetMap JSON into lines (downloads and bundled files alike)	When lines are loaded
`engine/RouteJson.kt` (36)	Saved form of a line (offline lines and test routes)	On save and load
`engine/TrackHmm.kt` (150)	Probability model of where on the line the train is, combining radio matches with travelled distance	On each radio scan while on a line

File	Job	When it runs
`engine/RadioMap.kt` (309)	Wi-Fi and cell fingerprints: learnt with GPS, matched without	Learning on every scan with GPS; matching on every scan without
`engine/RideMemory.kt` (161)	Learnt run speeds between stations and the vibration profile (still vs moving)	Loaded at start, learnt at each stop, saved at stop
`engine/Pdr.kt` (37)	Walking step length and direction	While walking off the line
`engine/PositionDeduper.kt` (33)	Keeps only new positions, plus one row every 30 s when standing	On each position about to be logged
`engine/TestDrive.kt` (63)	Test-drive states: recording, ready, tunnel	During a test drive
`engine/TestRouteRecorder.kt` (141)	Turns a GPS trip into a line: a point every 15 m, a station at each stop of 15 s or more	While recording a test route

Sensors

File	Job	When it runs
`sensor/SensorHub.kt` (431)	GPS, fused location (only when GPS is lost), accelerometer, gyroscope, orientation, barometer, step detector → a motion sample every 0.1 s	The whole time tracking is on
`sensor/RadioScanner.kt` (215)	Wi-Fi scan every 30 s; Wi-Fi RTT ranging every 2 s where supported	The whole time tracking is on
`sensor/CellScanner.kt` (180)	Serving and neighbour mobile cells every 5 s	The whole time tracking is on

File	Job	When it runs
`sensor/CellScanBuilder.kt` (61)	Turns cells into fingerprint readings, adding a handover marker when the serving cell changes	On each cell scan

Shared module `core`

File	Job
`models/Models.kt` (24)	`GpsPoint`: one location fix
`utils/GeoToolbox.kt` (50)	Distance and bearing between two points
`utils/NetworkStatus.kt` (39)	Internet state: connected, weak or none
`utils/TelemetryLogger.kt` (264)	A new timestamped log file per session, written in the background

Second app `gpslogger`

A plain GPS logger used for side-by-side comparison: `MainActivity.kt` (126) shows the list and buttons, `LoggerService.kt` (105) writes every fix, `LocationLog.kt` (46) keeps the on-screen list.

Data and tests

- `app/src/main/assets/metro/`: city line files bundled in the APK, so lines work with no network at all.
- `app/src/test/`: 104 tests run on the computer, no phone needed (`./gradlew test`).

Test file	What it checks
`UndergroundNavigatorTest`	Basic ride on a straight line: tracking, stops, holding at the next station, passing a station
`EdgeCaseTest`	GPS / network / sensor edge cases, learnt stillness
`UndergroundScenarioTest`	Long tunnels with no GPS and no network on a 20 km curved line
`RideSimulationTest`	End-to-end rides with realistic, noisy sensors

Test file	What it checks
`TunnelSimulator`	The physics ride simulator used by the two above (with Wi-Fi and cells)
`PositioningComponentsTest`	Radio map, track model, walking
`CellFingerprintTest`	Cell scans, handover markers, matching next to Wi-Fi
`TestDriveTest`	Recording a route and riding it without GPS
`MetroRouteTest`, `BundledLinesTest`	Line geometry and the bundled city files